

Scoring and leaderboard metrics

Quant Guild Trading Bot competition, IIT Madras

Odd semester 2026

Abstract

The board does not rank on profit. It ranks on a *doubly normalised* score: each block's profit is first divided by that block's hidden maximum value, and then standardised against the other nineteen bots that played the same block. This document defines that score and every figure reported beside it, gives the formula for each, and says which of them are ranking quantities and which are diagnostics. Section 7 covers the mock auctions, which run the same machinery but do not feed the final standing.

Contents

1	Notation	2
2	What happens inside a block	2
2.1	Values	2
2.2	Capital	3
2.3	Legality and bankruptcy	3
2.4	Payoffs	3
3	The ranking quantity	4
3.1	Step 1 — normalised block profit π	4
3.2	Step 2 — standardisation within the group	4
3.3	Step 3 — iteration score and total score	4
4	Every column on the board	5
4.1	score — Total score	5
4.2	mean_block_points — Mean block score	5
4.3	mean_normalised_profit — Mean π	5
4.4	normalised_profit_spread — Consistency	5
4.5	worst_normalised_profit — Worst block π	5
4.6	survival_rate — Survival	6
4.7	Raw profit columns	6
4.8	Counters	6
4.9	Disqualification	6
5	Ranking and tie-breaks	7
6	The tournament	7
7	Mock auctions	8
8	Reading the board: three worked cases	9

1 Notation

One *game* is a group of $N = 20$ bots playing $T = 2000$ rounds, split into $B = 4$ blocks of $L = 500$. One *iteration* is a fresh draw of groups across the whole field; a *showdown* is a set of iterations played back to back and published as one leaderboard.

Symbol	Meaning	Where it lives
i	a bot (index within its group)	<code>key</code>
$b \in \{1, \dots, B\}$	a block of L rounds	<code>BlockSummary.block</code>
t	a round, $1 \leq t \leq T$	<code>obs["round"]</code>
$[m_b, M_b]$	the block's hidden value bounds	<code>block_bounds</code> (secret)
$x_{i,t}$	bot i 's private value in round t	<code>obs["x"]</code>
n_t	players still solvent in round t	<code>obs["num_players"]</code>
X_t	max value over the <i>active</i> players	<code>max_value</code>
$b_{(1)} \geq b_{(2)} \geq \dots$	the round's bids, sorted	<code>sorted_bids</code>
$C_{i,t}$	bot i 's capital after round t	<code>Player.capital</code>
$C_{i,b}^{\text{start}}, C_{i,b}^{\text{end}}$	capital at the block's ends	<code>BlockSummary</code>

X_t is the maximum over the players *active that round*. A bankrupt bot draws no value, so it contributes nothing to X_t and is not counted in n_t . That is why n_t is handed to every bot: as the field thins, $\mathbb{E}[X_t]$ falls.

2 What happens inside a block

2.1 Values

Every active bot independently draws

$$x_{i,t} \sim \mathcal{U}[m_b, M_b], \quad (1)$$

with $[m_b, M_b]$ fixed within a block and redrawn at every boundary. Neither bound, nor the block index, nor the fact that a boundary has just happened, is ever reported to a bot. Detecting the regime change is part of the problem.

The bounds themselves are drawn, not chosen. Each block independently takes

$$m_b \sim \mathcal{U}\{10, 20, \dots, 1000\}, \quad r_b \sim \mathcal{U}\{100, 200, \dots, 10000\}, \quad M_b = m_b + r_b, \quad (2)$$

uniformly over the two grids — 100 values each, so 10^4 possible blocks. Both grids span two orders of magnitude, and two properties of them do real work:

- **m_b is never zero.** A bot cannot treat x as a draw from $\mathcal{U}[0, M]$ and read M off the maximum it has seen. The block $[1000, 1100]$, where every value sits within 10% of every other, is exactly as likely as $[10, 10010]$.
- **The width is independent of the floor.** Scale and spread move separately, so “how far above the floor am I” and “how close to the top am I” are different questions with different answers block to block.

Each *iteration* draws its own schedule of B blocks from the master seed, so no two iterations share one; every group inside an iteration plays the same schedule, which is what makes their standardised scores comparable and what a balanced or finals iteration seeds on. A hand-written schedule remains possible (`bounds_mode = "fixed"`) for reproducing a specific run.

2.2 Capital

Capital is redrawn independently for every bot at the start of every block:

$$\kappa \sim \mathcal{U}[0.5, 2.5], \quad \boxed{C_{i,b}^{\text{start}} = m_b + r_b \kappa} \quad (3)$$

with m_b and r_b that block’s floor and width from (2). What a bot finished the previous block with does *not* carry over.

Capital is anchored to the floor and scaled by the *width*, not by M_b . That is what keeps κ meaning one thing — “how many block-widths of headroom I start with” — across blocks that look nothing like each other. Scaling by M_b instead would hand every bot roughly the same bankroll relative to the spread of values in a narrow block like $[1000, 1100]$, which is the one quantity the capital draw exists to vary.

It also needs no patching. Since $m_b \geq 10$ and $r_b \geq 100$, the smallest possible capital is $10 + 100 \times 0.5 = 60$: comfortably positive, with no floor term and no additive jitter to keep a small block off zero.

Two consequences shape the whole score. First, within a block some bots start on half a block-width of headroom and some on two and a half, so a strategy that behaves identically on a thin bankroll and a fat one is being measured on exactly that. Second, because capital resets, the four blocks are four *independent* tests: a bot cannot bank a good block and coast.

2.3 Legality and bankruptcy

The legal bid ceiling is the bot’s own capital. A bid above it, negative, NaN, infinite, not a number, or not returned inside the per-round time limit is filed as a bid of 0 for that round.

Capital updates as $C_{i,t} = C_{i,t-1} + \text{payoff}$. A bot with $C_{i,t} \leq 0$ is eliminated *for the remainder of that block* and returns at the next boundary on a fresh draw from (3). Bankruptcy is therefore never fatal, but it forfeits the rest of the block, which is expensive in exactly the way Section 4.6 describes.

2.4 Payoffs

With $X = X_t$ and $b_1 \geq b_2 \geq \dots$ the sorted bids of the round:

Variation 1 — private value, first price

The winner takes $x_i - b_1$, using *the winner’s own* value; everyone else zero.

Variation 2 — common value, first price

The winner takes $X - b_1$; everyone else zero.

Variation 3 — runner-up penalty

$$\pi^{(1)} = X - b_1, \quad \pi^{(2)} = -\frac{1}{2} \max(X - b_1, 0),$$

and zero for everyone else. When the winner overpays the runner-up pays nothing: the penalty never becomes a reward for the winner’s loss.

Variation 4 — funded second price

If $b_1 \leq X$,

$$\pi^{(1)} = X - b_2, \quad \pi^{(2)} = X - b_1, \quad \pi^{(k)} = -\frac{w_k}{\sum_{j \in F} w_j} \left(\pi^{(1)} + \pi^{(2)} \right) \quad \text{for } k \in F,$$

with $w_3 = 0.5$, $w_4 = 0.3$, $w_5 = 0.2$ and $F = \{3, 4, 5\} \cap \{1, \dots, n_t\}$. The renormalisation is what keeps the round *exactly zero-sum* when fewer than five players are active; with $n_t \leq 2$ no penalty is collected. If instead $b_1 > X$, rank 1 alone takes $X - b_1$ and nobody else is touched — that branch is not zero-sum, by design.

Ties. Variations 1 and 2 let *every* bidder within 10^{-9} of the top bid win, each collecting the full payoff. Variations 3 and 4 need distinct ranks, so ties there are broken uniformly at random.

3 The ranking quantity

3.1 Step 1 — normalised block profit π

Raw profit is not comparable across blocks: a block with $M_b = 500$ pays out roughly fifty times one with $M_b = 10$, and inside a single block bots start from different capitals. So profit is measured in units of the block’s own hidden maximum:

$$\pi_{i,b} = \frac{C_{i,b}^{\text{end}} - C_{i,b}^{\text{start}}}{M_b} \quad (4)$$

with $\pi_{i,b} := 0$ in the degenerate case $M_b \leq 0$. A bot that went bankrupt ends the block on zero capital, so its $\pi_{i,b} = -C_{i,b}^{\text{start}}/M_b$, which by (3) is at worst about -2.5 .

π is the only quantity in the whole system with an absolute meaning. Everything downstream is relative. $\pi_{i,b} > 0$ means the strategy made money that block; $\pi_{i,b} < 0$ means it lost money, however well it placed.

3.2 Step 2 — standardisation within the group

Let $\mathcal{G}$ be the group that played block b together, $|\mathcal{G}| = N$. Write

$$\mu_b = \frac{1}{N} \sum_{j \in \mathcal{G}} \pi_{j,b}, \quad \sigma_b = \sqrt{\frac{1}{N} \sum_{j \in \mathcal{G}} (\pi_{j,b} - \mu_b)^2} \quad (5)$$

(the *population* standard deviation — `statistics.pstdev` — not the sample one; the twenty bots are the whole population of that block, not a sample from it). Then

$$z_{i,b} = \text{clip}\left(\frac{\pi_{i,b} - \mu_b}{\sigma_b}, -3, +3\right), \quad P_{i,b} = 50 + 15 z_{i,b} \quad (6)$$

so $P_{i,b} \in [5, 95]$, centred on 50.

Two edge cases are defined explicitly. If $\sigma_b = 0$ — every bot in the group scored identically — then $z_{i,b} := 0$ and everyone gets 50; nobody is separated by a block that separated nobody. If the group has a single member, that member gets 50.

The ± 3 clip is not cosmetic. Without it a single block in which one bot happened to draw a large κ and win a run of rounds could contribute several hundred points and decide the competition on variance rather than skill. Clipping caps any one block’s contribution at 95 and floors it at 5.

3.3 Step 3 — iteration score and total score

$$S_i^{\text{iter}} = \sum_{b=1}^B P_{i,b}, \quad \text{score}_i = \sum_{\text{iterations } r} S_{i,r}^{\text{iter}} \quad (7)$$

score is the ranking quantity, and the only one. With $B = 4$ blocks per iteration and points centred on 50, an average bot scores about 200 per iteration; over a three-iteration showdown, about 600.

Because `score` is a *sum*, a bot that played fewer iterations than the rest scores lower purely for that. This is what `mean_block_points` exists to correct, and it is the documented tie-break (Section 5).

4 Every column on the board

Below, $\mathcal{B}_i$ is the multiset of all blocks bot i played across the whole showdown ($|\mathcal{B}_i| = \text{blocks}$) and $\mathcal{G}_i$ the set of games ($|\mathcal{G}_i| = \text{games}$). All of these are computed in `harness/evaluate.py:aggregate`.

4.1 `score` — Total score

Equation (7). Higher is better. This is what the board sorts on.

4.2 `mean_block_points` — Mean block score

$$\bar{P}_i = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} P_{i,b} \in [5, 95] \quad (8)$$

`score` normalised by how much a bot actually played. It is the tie-break, and it is the honest number to quote when two bots played different numbers of games. Read it against 50: above means the bot beat its group’s average block, below means it did not.

4.3 `mean_normalised_profit` — Mean π

$$\bar{\pi}_i = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \pi_{i,b} \quad (9)$$

The absolute-profitability check. Standardisation makes P a purely relative statement: in a group where every bot loses money, somebody still scores 95. $\bar{\pi}$ is the answer to “but did it actually make money?” — expressed in units of M_b , so $\bar{\pi} = +0.4$ means the strategy earned, on average, forty percent of the block’s maximum value.

A bot can rank first on `score` with $\bar{\pi} < 0$. That is not a contradiction and not a bug: it means the bot lost the least in a field that all lost. It is, however, exactly the situation a report should notice. In variations 3 and 4 it is common: those are transfer games, so the group’s total π cannot be positive in the branch where the winner did not overpay.

4.4 `normalised_profit_spread` — Consistency

$$s_i = \sqrt{\frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} (\pi_{i,b} - \bar{\pi}_i)^2} \quad (10)$$

The population standard deviation of π across *all* blocks the bot played, in every game. **Lower is better**, and the board sorts this column ascending for that reason.

This is the column that separates a strategy from a lottery ticket. Two bots can share a $\bar{\pi}$ of $+0.3$ with one earning $(0.3, 0.3, 0.3, 0.3)$ and the other $(1.5, -0.4, 0.3, -0.2)$; they are not the same bot, and the second one is one bad draw from bankruptcy. Note also that a large s_i is often the signature of a bot that has *not* detected the block boundary: it is tuned to one regime and is being carried by the blocks that happen to match it.

The four blocks in an iteration differ deliberately in both scale and width, so s_i measures robustness to the regime change, not just round-to-round noise.

4.5 `worst_normalised_profit` — Worst block π

$$\pi_i^{\min} = \min_{b \in \mathcal{B}_i} \pi_{i,b} \quad (11)$$

The tail, reported directly. s_i can be dragged up by an unusually *good* block, which is not a risk; $\pi_i^{\min}$ cannot. A bot whose $\pi_i^{\min}$ approaches $-C_{i,b}^{\text{start}}/M_b$ (Section 2.2) lost its entire starting capital in some block, which is the same statement as `survival_rate` < 1 .

4.6 survival_rate — Survival

$$\rho_i = \frac{\#\{b \in \mathcal{B}_i : \text{not bankrupt at the end of } b\}}{|\mathcal{B}_i|} \quad (12)$$

The fraction of blocks the bot finished solvent; 1 when it never went broke. A block counts as survived when $C_{i,b}^{\text{end}} > 0$ at the block’s last round — equivalently, when the bot was never eliminated during it.

Survival is a *diagnostic, not a penalty term*: it appears nowhere in (7). It does not need to. Going bankrupt forfeits the remainder of the block, so those rounds earn nothing while the rest of the group keeps compounding, which drives $\pi_{i,b}$ to its floor and $P_{i,b}$ towards 5. The scoring penalises bankruptcy through π ; ρ_i just tells you it happened, and how often.

Read ρ_i together with $\bar{\pi}_i$. A bot with $\bar{\pi} > 0$ and $\rho < 1$ is one that funds good blocks with catastrophic ones — the profile most likely to collapse when the hidden bounds change to ones it has never seen.

4.7 Raw profit columns

Reported for context, and never used to rank. With $\Delta_{i,g}$ the net profit of bot i in game g (the sum of $C^{\text{end}} - C^{\text{start}}$ over that game’s blocks):

$$\text{mean_net_profit} = \frac{1}{|\mathcal{G}_i|} \sum_g \Delta_{i,g}, \quad \text{std_net_profit} = \sqrt{\frac{1}{|\mathcal{G}_i|} \sum_g (\Delta_{i,g} - \bar{\Delta}_i)^2}, \quad (13)$$

$$\text{best_net_profit} = \max_g \Delta_{i,g}, \quad \text{worst_net_profit} = \min_g \Delta_{i,g}, \quad (14)$$

and $\text{mean_final_capital}$ is the mean of C^{end} over the last block of each game. These are in raw currency units and are therefore dominated by whichever blocks happened to have the largest M_b ; that is precisely why they do not rank anybody.

4.8 Counters

Column	Definition	Note
games	$ \mathcal{G}_i $	group-games played this showdown
blocks	$ \mathcal{B}_i $	normally $B \times \text{games}$
wins	rounds finishing rank 1	in V1/V2 every tied top bidder wins
errors	<code>get_bid</code> raised, or returned a non-number, NaN, $\pm\infty$ or < 0	each filed as a bid of 0
timeouts	rounds over the time limit	each filed as a bid of 0

An over-capital bid is *not* counted as an error: it is legal to attempt and is simply filed as 0. Only malformed returns are counted, which is why a bot can bleed capital through zero-bids with **errors** sitting at 0.

wins is a diagnostic and a deliberately misleading one if read alone: in variation 1 a bot that wins every round by overbidding has a large **wins** and a deeply negative π . In variation 4 rank 1 and rank 2 are *both* paid, so **wins** counts only half the times the bot was in the money.

4.9 Disqualification

disqualified is set by the sandbox, not by the scoring: a policy violation, a memory kill, or repeated failure to return. A disqualified bot keeps its computed row — the numbers stay visible — but always sorts last, below every bot that finished legally.

5 Ranking and tie-breaks

Within each variation, rows are ordered by the key

$$\left(\mathbb{K}[\text{disqualified}], \text{--score}, \text{--mean_block_points} \right) \quad (15)$$

ascending. In words:

1. every legal bot outranks every disqualified one;
2. then by total score, highest first;
3. ties broken by mean block score, highest first — which is what stops a bot that played fewer games from being punished for the scheduler’s arithmetic rather than its own play.

The four variations are ranked on four separate boards. There is no combined score across variations; a bot entered in one variation is not comparable to a bot entered in another, because π is standardised against a different field.

6 The tournament

A single 20-bot arrangement is far too noisy to rank on — who you are grouped with matters as much as how you play. In variations 3 and 4, where payoffs move between players rather than being created by the auction, it matters more still. So each variation is played over several iterations.

Iteration	Grouping	What it does
1	random	the field shuffled into groups of 20 under seed S_1
2	random, fresh seed	reshuffled independently: new distributions, capitals, opponents
3	strength-balanced	snake-seeded by cumulative points (below)
4–5	finals	the top <code>finals_size</code> by cumulative score, head to head

Snake seeding. Bots are sorted by cumulative score and dealt into G groups in a serpentine: ranks $1 \dots G$ go to groups $1 \dots G$, ranks $G+1 \dots 2G$ go back down $G \dots 1$, and so on. Formally, the bot at sorted position p (0-based) with $(r, c) = \text{divmod}(p, G)$ lands in group

$$g(p) = \begin{cases} c & r \text{ even} \\ G - 1 - c & r \text{ odd.} \end{cases} \quad (16)$$

This *balances* groups rather than segregating them: every group ends up with roughly the same average strength, which is what makes standardised scores comparable across groups.

Remainders. Under random grouping, a trailing group smaller than half of N is folded into the previous group rather than left to play mostly against filler bots. Snake seeding needs no such rule: it deals every bot into an existing group by construction.

Filler bots. A group short of N players is topped up with the three published sample bots so that the highest-bid race is a real one. Fillers play for real but never appear on a leaderboard and never contribute a row.

Independence. Each iteration uses its own block schedule $\{[m_b, M_b]\}$, not merely the same schedule under a new seed, so iteration 2 is a genuinely fresh test rather than a re-roll of the same one. Groups within an iteration are independent games and are run in parallel.

Final ranking is the sum of the two finals iterations, tie-broken by the qualification total.

How this maps onto a run. `grouping` is set *per iteration*, so the table above is one run: `["random", "random", "balanced", "finals", "finals"]`. Iterations play in order, and each one seeds on the standing after the ones before it — a balanced iteration snake-seeds on iterations 1–2’s points, and a finals iteration cuts the field on everything above it. A run whose iterations are all `random` needs no standing at all and does not consult one.

A schedule shorter than `iterations` *holds its last entry* rather than cycling. That is what the default, `["random", "balanced"]`, relies on: iteration 1 draws groups blind because there is nothing to seed on yet, and every iteration after it is strength-balanced on the standing so far, however many there are. So the tournament runs in full on *every* showdown — practice, mock and final alike — rather than only when someone remembers to set it up.

Where a run needs a standing it did not produce itself — the first balanced iteration of a fresh board — it takes the last finished showdown *of its own kind* (Section 7), never whichever board happened to finish most recently.

7 Mock auctions

Two mock showdowns run before the deadline. They use *the same engine, the same sandbox, the same scoring, the same aggregation and the same tournament structure* as the graded run — there is no separate code path, and a mock is not a reduced format. It plays the full sequence: iteration 1 on blind groups, every iteration after it strength-balanced on the standing so far. They differ only in size, in what is done with the result, and in one recorded fact: every showdown is stamped with what it was for.

Kind	What it is	Board
<code>practice</code>	the clock, every <code>interval_minutes</code>	rewritten by the next tick
<code>mock</code>	an announced mock auction	archived and published
<code>final</code>	the run after the deadline	the result

The stamp is not a label. A `balanced` or `finals` iteration seeds on a standing, and where that standing comes from decides who plays whom: seeding takes the last finished run *of the same kind*, so a mock is seeded by the previous mock and never by a practice run against throwaway bounds. It is also what keeps a mock board reachable after the two-hourly clock has replaced the live one.

	Mock auction 1	Mock auction 2	Final evaluation
Variations	1, 2	1, 2, 3, 4	1, 2, 3, 4
Submissions	optional	optional	required
Grouping	the full tournament: iteration 1 random, the rest seeded		
Iterations	reduced	reduced	3 qualification + 2 finals
Bounds	drawn from the run's own seed — never reused		
Counts towards the result	no	no	yes
Also does	releases variations 3 and 4 (mock 1)		

Four things follow, and they are the whole point of running them.

1. **A mock score is not a partial final score.** Nothing carries over. The graded run starts from an empty board, on block bounds and seeds that have never been published, against whatever field submits by the deadline. A mock rank is a measurement, not a position.
2. **The hidden bounds are different, deliberately.** Every iteration of every run draws its own blocks from (2), off a seed that is not published until afterwards. There are 10^4 possible blocks and no schedule is ever reused, so a bot tuned to the π it earned in a mock is tuned to a distribution that will not appear again — the same trap the local runner warns about, one level up. This is why the bounds are no longer the competition's principal secret: the seed is.
3. **The per-block table is the deliverable, not the rank.** Each participant is sent their $(C_b^{\text{start}}, C_b^{\text{end}}, M_b, \pi_b, P_b, \text{bankrupt})$ row for every block. That is where a bot's failure mode is legible: a π_b that collapses in exactly one block is a bot that has not detected the boundary; a

low P_b against a healthy π_b is a bot that is profitable but being out-played; a bankruptcy tells you which block and which round.

4. **Mock 1 is also the release gate.** The last two variations are published the moment mock auction 1's results are, so their mock field necessarily has fewer entrants and less-developed bots than the final one. Read a mock 2 rank in a late-released variation with that in mind: the standardisation in (6) is against *that* field, and a thin field of under-tuned bots makes σ_b small and z — and so the whole board — unusually sensitive.

Operationally, a mock run is an ordinary showdown triggered from the admin console and stamped **mock**; the practice board that rewrites itself every two hours is the same thing again, stamped **practice**, at a smaller size. Nothing on either is graded. The only run that is graded is the one after the final deadline, on settings that are not published until it has finished.

8 Reading the board: three worked cases

A bot ranked 3rd with $\bar{\pi} = -0.15$. It is losing money in absolute terms and still beating most of its group. In V1 or V2 that says the whole field is overbidding and the auction is dissipating its own surplus. In V3 or V4 it is unremarkable: those are transfer games, and the group's total π cannot be positive in the branch where the winner did not overpay. Check ρ_i : if it is 1, the bot is simply being conservative in a bloodbath, which is a perfectly good strategy.

A bot with $\bar{P} = 62$ and $s = 1.1$. Strong on average, wildly inconsistent. With four blocks per iteration, an s of 1.1 against a $\bar{\pi}$ of a few tenths means individual blocks are swinging by more than the block's own maximum value. Look at $\pi^{\min}$ and ρ : this is usually a bot that is excellent in one regime and bankrupt in another, and it will not survive a fresh block schedule.

A bot with $\rho = 0.75$ and rank 1. It went bankrupt in a quarter of its blocks and still tops the board, so the blocks it survived were dominant enough to absorb three P -values near the floor. That is a real result, but it is a 3σ -clipped one: the clip in (6) caps how much its good blocks can earn while its bad ones sit at 5. It is close to the point where one more bad draw flips the ranking, and it is the first bot to read by hand.